UAV Software Engineering: C++ Memory Management

Smart Pointers e RAII

1 RAII paradigm

In the critical UAV context memory leak is just unacceptable. Modern C++ uses RAII (Resource Acquisition Is Initialization) to mitigated the problem. Memory allocation happens in the object constructor and the deallocation happens in the distructure. When the object goes out of scope, the memory get automatically deallocated, granting a fixed execution time (which is around 400Hz in the loops).

2 `std::unique_ptr` and Ownership

`std::unique_ptr` it grants the exclusive ownership of a memory block allocated on the heap. Note that it doesn't add any sort of overhead over the standard allocation and destruction using `new` and `delete` (raw pointer). Since the ownership is exclusive, an `unique_ptr` can not be copied, but only moved using `std::move()`. This explicit ownership transfer prevents hidden bugs such as *dangling references* or *double free*, which are common on legacy systems.

3 Implementation Details: `make_unique` and Observers

Modern C++ (C++14 and later) strictly recommends using `std::make_unique` rather than calling `new` directly. This approach guarantees exception safety; if an exception is thrown during object construction, the memory is automatically reclaimed, eliminating a subtle source of leaks.

While `unique_ptr` enforces exclusive ownership, other parts of the drone's system may need to read the data without taking ownership. The `.get()` method exposes the underlying raw pointer safely, acting strictly as an observer. Additionally, the ownership state can be verified through implicit boolean evaluation (e.g., `if (!ptr)`), which is critical to check after a `std::move()` operation to ensure the original pointer is now null and safe from accidental use.